Documentation technique — ML Churn Project

Version du modèle : 1.0.0 · Auteur : Deschant Kounou

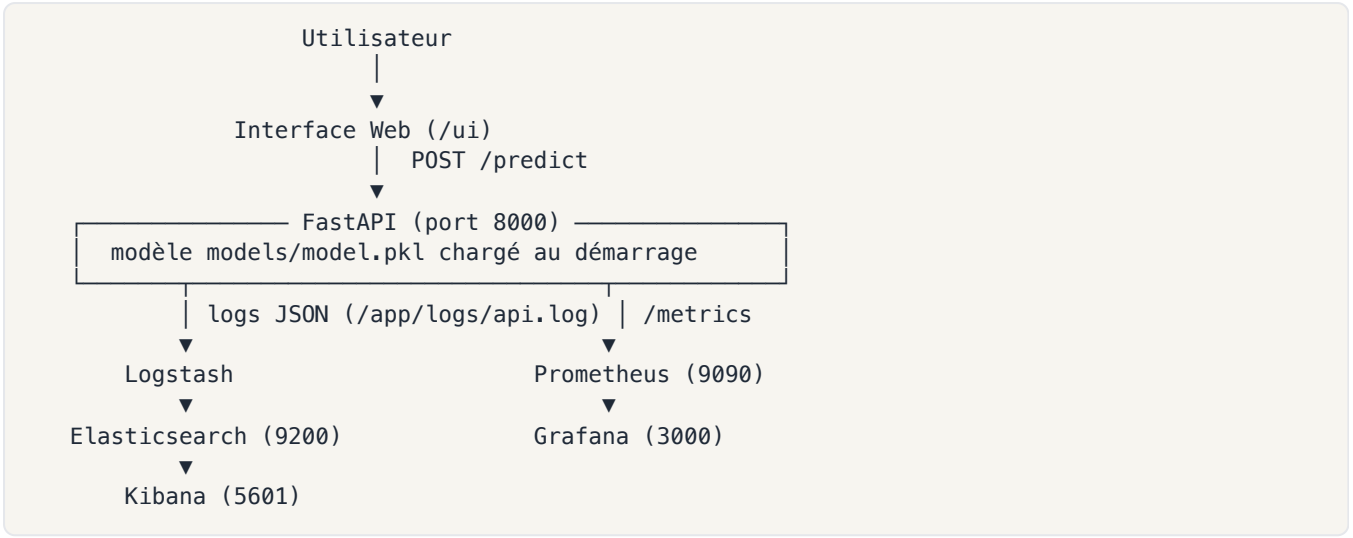
Ce document décrit le fonctionnement interne de l'application. L'installation et les commandes de lancement sont dans le README.

---

1. Vue d'ensemble

L'application prédit si un client d'un opérateur télécom risque de résilier son abonnement (churn). Elle se compose de :

Brique	Technologie	Rôle
Données	pandas, numpy	Génération d'un dataset synthétique de 2 000 clients
Modèle	scikit-learn	Random Forest de 200 arbres, sérialisé avec joblib
API	FastAPI + Uvicorn	Exposition du modèle en HTTP/JSON
Interface	HTML + JavaScript	Formulaire de saisie client, servi par l'API sur `/ui`
Tests	pytest, Selenium, JMeter	Tests API, tests E2E navigateur, tests de charge
Conteneurs	Docker, Docker Compose	Image de l'API et orchestration de la stack
CI	GitHub Actions	Tests, E2E et build Docker à chaque push / pull request
Logs	Logstash, Elasticsearch, Kibana	Collecte, indexation et recherche des logs
Métriques	Prometheus, Grafana	Collecte et tableau de bord des métriques



---

2. Données

Script : `src/generate_data.py` → `data/raw/churn.csv` (graine aléatoire fixée à 42, donc reproductible).

Variable	Type	Plage	Description
`age`	entier	18 – 69	Âge du client
`anciennete`	entier	1 – 119	Ancienneté en mois
`revenu`	entier	150 000 – 999 999	Revenu
`nb_reclamations`	entier	loi de Poisson (λ = 2)	Nombre de réclamations
`montant_mensuel`	entier	5 000 – 74 999	Montant de l'abonnement
`retards_paiement`	entier	loi de Poisson (λ = 1)	Nombre de retards de paiement
`churn`	0 / 1	—	Cible : 1 = le client part

La cible est tirée d'une probabilité logistique qui augmente avec les réclamations, les retards et le montant, et baisse avec l'ancienneté. `age` et `revenu` n'influencent pas la cible.

Répartition obtenue : 1 139 clients restent (57 %), 861 partent (43 %). L'exploration (`notebooks/exploration.py`) produit `data/churn_distribution.png` et `data/correlation.png`.

---

3. Modèle

Script : `src/train.py`. Découpage 80 % entraînement / 20 % test, stratifié sur `churn`, `random_state=42`.

Résultats sur le jeu de test (400 clients) :

Modèle	Accuracy	Precision	Recall	F1
Régression logistique (avec StandardScaler)	0,625	0,582	0,453	0,510
Arbre de décision (profondeur 5)	**0,632**	**0,583**	**0,512**	**0,545**
Random Forest (200 arbres)	0,600	0,543	0,436	0,484

Le sujet impose le Random Forest pour la suite du TP : c'est lui qui est sauvegardé dans `models/model.pkl`. Sur ce dataset, il est pourtant le moins bon des trois (voir le rapport d'industrialisation, section « Limites »).

Le modèle attend les 6 variables **dans l'ordre** du tableau de la section 2. L'API construit un `DataFrame` avec les noms de colonnes, ce qui évite les erreurs d'ordre et les avertissements de scikit-learn.

---

4. API

Fichier : `api/main.py`. Le modèle est chargé une seule fois, au démarrage.

Méthode	Route	Réponse
GET	`/`	Nom, version et statut de l'application
GET	`/health`	`{"status": "UP"}` (utilisé par la CI et les tests E2E)
GET	`/model/info`	`{"model": "Random Forest", "version": "1.0.0"}`
POST	`/predict`	Prédiction pour un client
GET	`/ui`	Interface web (`frontend/index.html`)
GET	`/metrics`	Métriques au format Prometheus
GET	`/docs`	Documentation Swagger générée par FastAPI

`POST /predict`

Requête :

```
{"age": 35, "anciennete": 5, "revenu": 450000, "nb_reclamations": 2,
 "montant_mensuel": 25000, "retards_paiement": 2}
```

Réponse `200` :

```
{"prediction": "CHURN", "probability": 0.67, "model_version": "1.0.0", "duration_ms": 15.87}
```

- `probability` est la probabilité de churn. `prediction` vaut `CHURN` si le modèle prédit la classe 1, sinon `NO_CHURN`.
- `422` : corps invalide (champ manquant ou mauvais type). La validation est faite par Pydantic (`ClientData`).
- `500` : erreur pendant l'inférence. Elle est journalisée avec la trace et comptée dans `churn_prediction_errors_total`.

---

5. Logs

Chaque ligne de log est un objet JSON (classe `JsonFormatter`). Les logs vont sur la sortie standard et, si la variable `LOG_FILE` est définie, dans un fichier. Dans Docker Compose, `LOG_FILE=/app/logs/api.log` est écrit sur le volume `api-logs`, partagé avec Logstash.

Champ	Question à laquelle il répond
`timestamp`	Quand ?
`client_ip`	Qui ?
`operation` (`prediction`, `prediction_error`)	Quelle opération ?
`model`, `model_version`	Quel modèle ?
`input`, `prediction`, `probability`	Avec quelles données, pour quel résultat ?
`response_time_ms`	Combien de temps ?
`level`, `exception`	Quelle erreur ?

Le champ s'appelle `operation` et non `event` : Logstash crée lui-même un objet `event` (`event.original`), et un champ texte du même nom fait échouer le décodage JSON (`_jsonparsefailure`).

Pipeline Logstash

Fichier : `docker/logstash/logstash.conf`.

1. **input** : lit `/var/log/api/*.log` et décode chaque ligne en JSON.
2. **filter** : utilise `timestamp` comme date de l'événement (`@timestamp`), supprime les champs inutiles.
3. **output** : envoie dans Elasticsearch, un index par jour `ml-churn-logs-AAAA.MM.JJ`.

Exemples de recherches dans Kibana (Data View `ml-churn-logs-*`) : `operation:prediction`, `level:ERROR`, `prediction:CHURN`, `model_version:"1.0.0"`, `response_time_ms > 100`.

---

6. Métriques

Exposées sur `/metrics`, collectées par Prometheus toutes les 5 secondes (`monitoring/prometheus/prometheus.yml`, cible `api:8000`).

Métrique	Type	Source
`http_requests_total{handler, method, status}`	Compteur	prometheus-fastapi-instrumentator
`http_request_duration_seconds`	Histogramme	prometheus-fastapi-instrumentator
`churn_predictions_total{result, model_version}`	Compteur	Application
`churn_prediction_errors_total{model_version}`	Compteur	Application
`churn_prediction_duration_seconds{model_version}`	Histogramme	Application (temps d'inférence seul)

Tableau de bord Grafana

La source de données et le tableau de bord sont chargés automatiquement au démarrage (`monitoring/grafana/provisioning/` et `monitoring/grafana/dashboards/ml-churn-dashboard.json`).

Panneau	Requête PromQL
Nombre de requêtes	`sum by (handler) (rate(http_requests_total{handler!="/metrics"}[1m]))`
Temps de réponse p50 / p95	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{handler="/predict"}[1m])))`
Nombre d'erreurs	`sum(increase(http_requests_total{status=~"[45]xx"}[$__range]))`
Taux d'erreur	`sum(rate(http_requests_total{status=~"5xx"}[1m])) / sum(rate(http_requests_total[1m]))`
Nombre de prédictions	`sum(churn_predictions_total)`
Prédictions par résultat	`sum by (result) (churn_predictions_total)`

7. Tests

Fichier	Type	Ce qui est vérifié
`tests/test_api.py`	Tests API (TestClient, sans serveur)	Les routes répondent, la prédiction a le bon format, un corps invalide renvoie 422, `/ui` et `/metrics` fonctionnent
`tests/test_selenium.py`	E2E (Chrome headless)	La page Swagger s'ouvre ; parcours complet : saisie client → clic sur « Prédire » → résultat affiché et probabilité entre 0 et 1
`tests/jmeter/churn_load_test.jmx`	Charge	`POST /predict` sous 100, 500 et 1 000 utilisateurs ; chaque réponse doit avoir le code 200

Les tests Selenium portent le marqueur `e2e`. Ils sont **ignorés** (et non en échec) si l'API ne répond pas sur `APP_URL` (par défaut `http://localhost:8000`). `pytest -m "not e2e"` lance uniquement les tests API.

Le plan JMeter est paramétrable sans l'ouvrir : `-Jthreads`, `-Jrampup`, `-Jloops`, `-Jhost`, `-Jport`.

---

8. Conteneurs

Image de l'API (`Dockerfile`)

Base `python:3.14-slim`, installation de `requirements.txt`, copie du projet, lancement d'Uvicorn sur `0.0.0.0:8000`. Le modèle `models/model.pkl` est inclus dans l'image. `.dockerignore` exclut l'environnement virtuel, Git, les caches, les logs et les résultats JMeter.

Stack (`docker-compose.yml`)

Service	Image	Port	Dépend de
`api`	construite localement	8000	—
`elasticsearch`	elasticsearch 8.17.0 (nœud unique, sécurité désactivée, 512 Mo de mémoire Java)	9200	—
`logstash`	logstash 8.17.0	—	elasticsearch en bonne santé
`kibana`	kibana 8.17.0	5601	elasticsearch en bonne santé
`prometheus`	prom/prometheus v3.1.0	9090	api
`grafana`	grafana/grafana 11.4.0	3000	prometheus

Volumes : `api-logs` (logs partagés entre l'API et Logstash), `es-data`, `grafana-data`.

---

9. Intégration continue

Fichier : `.github/workflows/ci.yml`, déclenché par un push ou une pull request sur `main` et `develop`.

```
test (pytest -m "not e2e")
  ├──► e2e    : démarre Uvicorn, attend /health, lance tests/test_selenium.py
  └──► docker : construit l'image, lance le conteneur, appelle /health puis /predict
```

`e2e` et `docker` ne démarrent que si `test` a réussi. La CI utilise Python 3.14, comme le Dockerfile, pour que les versions figées dans `requirements.txt` s'installent pareil partout.

Pousser une modification de ce fichier demande un jeton GitHub avec le droit `workflow`.

---

10. Arborescence

```
ml-churn-project/
├── .github/workflows/ci.yml
├── api/main.py
├── data/raw/churn.csv
├── docker/logstash/logstash.conf
├── docs/
├── frontend/index.html
├── models/model.pkl
├── monitoring/
│   ├── prometheus/prometheus.yml
│   └── grafana/{provisioning/, dashboards/}
├── notebooks/exploration.py
├── src/{generate_data.py, train.py}
├── tests/{test_api.py, test_selenium.py, jmeter/}
├── Dockerfile
├── docker-compose.yml
├── pytest.ini
└── requirements.txt
```